

Synthetic Source Model

PREPARED BY Energy Hippo implementation team

The synthetic source database is the authoritative model for fake customer data. Ingestion files and integration payloads should be generated from this model rather than edited independently.

Core Entities

Entity	Purpose
customer	Synthetic customer identity and global settings.
site	Campus, apartment complex, plant, or other customer location grouping, owned by one EEM company.
building	Physical building, or an outdoor service area such as grounds or a carport with no floor area, with use type, size, address, and weather mapping.
energy_star_property	ENERGY STAR Portfolio Manager property mapping for a site or building (primary function, gross floor area, occupancy). Grounds have none.
property_use	One ENERGY STAR property use on a property, such as Office or Multifamily Housing.
property_use_detail	One use detail value on a property use, such as Weekly Operating Hours or Number of Bedrooms.
provider	Utility provider identity, including its kind and billing model.
provider_service	One commodity a provider supplies (a provider can supply several).
account	Provider account assigned to a site or building.
meter	Utility meter on a provider account, or an owned meter with no account: an apartment, house or sewer-deduct submeter behind a utility meter, a plant production meter, a sensor group, a logical source, or a plant controller.
channel	Measured stream on a meter, such as kWh, kW, therms, gallons, or chilled water tons.
reference_channel	External source stream that belongs to no building: the KSAC and KSMF weather stations. Building sensors, historians, controllers and test signals are owned meters with channels.
gateway_profile	One EEMSuite gateway component configuration to prove, such as AcquiSuite, MV90/MDEF, MV90/MV9, or BACnet.
gateway_node	Runtime gateway node/device context used by the EEMSuite gateway components.
gateway_point	Runtime point binding that maps an EEMSuite PtID to a synthetic source channel.
bill	Billing period, usage, cost, demand, and charge details.
bill_charge_line	One charge component on a bill (fixed, customer, energy, demand, tier, TOU, seasonal, rider, surcharge, tax, minimum, maximum, credit, adjustment) with quantity, unit, amount, optional rate-component link, and GL coding.
interval_reading	Time-series reading for one channel.
digital_reading	Status/state time-series reading for a digital point (an enumerated state, not a measured quantity).

Entity	Purpose
rate_schedule	Synthetic tariff metadata used to explain bills and rate changes.
scenario_event	A deliberate event such as a meter replacement, estimated bill, correction, duplicate file, or missing interval block.
note	Free-text note attached to a source entity (bill, meter, account, building).
image	Image or document attached to a source entity (bill image, meter photo, document).
artifact_run	A record of generated files and payloads for a scenario/version.
expected_assertion	Expected downstream result used by validation checks.

Stage 1 App Setup Entities

These entities are EEM setup targets rather than provider source data. They are kept separate from bills, intervals, and gateway runtime records so Stage 1 can prove a blank system setup through the apps before fake consumption begins.

Entity	Purpose
eem_company	EEM company/security boundary that should be created for the customer, such as a campus operations company, an operating company, or an internal service company.
eem_hierarchy_level	Company hierarchy level setup, including level order, EEM internal level type, user-defined field label, and DBAdmin FontAwesome class.
eem_system_context	Existing EEM system company context, currently CompanyID = -1, used for implementation/support access without creating a customer company.
eem_permission_profile	Semantic permission intent that will later be mapped to concrete EEM permissions or screens.
eem_security_group	Company-scoped group carrying one or more permission profiles.
eem_user	Fake person who can authenticate or, for disabled-user tests, is expected not to authenticate.
eem_group_membership	User membership in a company group. Direct user permissions are intentionally avoided.
eem_utility_provider_target	EEM utility/provider setup target, including commodity, rate schedule shells, and company usage.
eem_billing_account_shell	EEM account, cost center, or service agreement shell created before any bills are loaded.
eem_meter_target	EEM meter node target mapped to a source meter, logical meter, weather station, sensor group, or plant controller.
eem_point_target	EEM point node target mapped to a source channel, reference channel, aggregate, baseline, weather index, or derived register point.
eem_weather_station_assignment	Mapping from a location node to an EEM weather station meter node.
eem_aggregate_definition	EEM aggregate point plus member points, multipliers, required flags, and interval/unit compatibility rules.
eem_baseline_definition	EEM baseline point shell plus related measured point and index inputs such as HDD/CDD.

Entity	Purpose
eem_company_calendar	Fiscal calendar periods per EEM company (period year/month and boundaries) that gate bill processing, posting, and import validation. A deliberately omitted period is a negative-test fixture.
eem_permission_grant	Concrete permission grant: a company group's access to an EEM permission object (type <code>\$/A/D/EX/F/MI/MU/TS</code>) at an <code>AccessLevel</code> , optionally scoped to a hierarchy node. The loadready realization of <code>eem_permission_profile</code> .

Required Invariants

- Every generated customer, site, building, account, meter, and channel must have a stable synthetic identifier.
- Source identifiers must not look like real customer account or meter numbers unless clearly prefixed as synthetic.
- Timestamps must declare their time zone behavior.
- Bills must preserve billing period boundaries and provider units.
- Interval data must preserve source interval length and unit.
- ENERGY STAR `primary_function` and property use `function` values must be exact `PrimaryFunctionType` enum values, and each use detail must be valid for that property type.
- Gateway points must target a real meter channel or reference channel and preserve the EEMSuite runtime keys consumed by their component.
- A channel may be bound to more than one gateway profile only when those profiles are documented as alternate proof paths, not loaded together into the same baseline.
- A source site, building, meter, bill, or point must belong to one EEM company in the baseline; cross-company access should be modeled through group membership, not duplicated hierarchy.
- Stage 1 hierarchy levels must include their EEM internal level number and a DBAdmin-compatible FontAwesome class.
- Stage 1 app setup users must receive access through company groups, not direct user-level permission grants.
- Disabled users may remain in source memberships only if validation confirms they cannot authenticate.
- EEM meter and point targets may be created before bills or archive rows, but they must declare whether they are physical, logical, weather, aggregate, baseline, or derived points.
- Weather station setup must identify the weather meter, temperature point, related HDD/CDD points, and assigned locations.
- Aggregate members must match the aggregate interval and unit unless a documented EEM product rule supports the exception.
- Baseline points must match the measured point's unit and interval and use an explicit related-point relationship.
- Scenario events must identify the data they affect.
- Assertions must be traceable to scenario names and source identifiers.
- A bill's `bill_charge_line` amounts must sum to the bill total, and each line's quantity must reconcile to the bill's usage/demand for that commodity.
- Each `bill_charge_line` must declare a `charge_type`; lines derived from a rate construct should reference the determinant/component, and AP-exportable lines should carry (or default to) a GL account.

- eem_company_calendar must cover every fiscal period in which a bill is posted; any omitted period must be flagged as a negative-test fixture and referenced by the scenario that depends on it.
- eem_permission_grant rows must resolve to a real EEM permission object and a valid access_level; permission_type = D node-scoped grants must reference a hierarchy node in the same company; approval-capable grants must use access_level = 3; users still receive access only through group membership, never direct grants.
- Digital points must declare an enumerated state_set; digital_reading rows carry a state_value from that set, never a measured quantity.
- note and image rows must reference an existing source entity by table code + id.

Suggested Tables

The first implementation does not need to use these exact names, but it should preserve these concepts.

```
customer(
  customer_id,
  display_name,
  seed,
  time_zone
)

site(
  site_id,
  customer_id,
  display_name,
  site_type,
  region,
  address_line1,
  city,
  state,
  postal_code,
  latitude,
  longitude,
  coordinate_source
)

building(
  building_id,
  site_id,
  display_name,
  use_type,
  gross_floor_area_sqft,
  address_line1,
  city,
  state,
  postal_code,
  latitude,
  longitude,
  coordinate_source,
  primary_weather_station
)

energy_star_property(
  energy_star_property_id,
  site_id,                -- set for campus_parent / multifamily_property roles
  building_id,            -- set for child_property / building_within_property / standalone_property
  role,                   -- campus_parent | child_property | multifamily_property | building_within_
                           property | standalone_property
)
```

```

primary_function,      -- exact ENERGY STAR PrimaryFunctionType value, e.g. College/University, Office
fice
gross_floor_area,
gross_floor_area_units, -- Square Feet | Square Meters
year_built,
occupancy_percentage,  -- rounded to ENERGY STAR 5% increments on export
is_institutional,
parent_site_id        -- campus/property this child rolls up to
)

property_use(
  property_use_id,
  energy_star_property_id,
  use_function,        -- ENERGY STAR property use type, e.g. Office, Laboratory, Multifamily Housing
  gross_floor_area_sqft
)

property_use_detail(
  property_use_detail_id,
  property_use_id,
  detail_name,         -- e.g. weeklyOperatingHours, totalNumberOfResidentialLivingUnits, resident
  PopulationType
  detail_value,
  detail_units,        -- where applicable, e.g. Square Feet
  temporary,          -- provisional flag (ENERGY STAR UseAttributeBase)
  current_as_of       -- date the value became current
)

provider(
  provider_id,
  display_name,
  identity_class,      -- everyday | unique
  provider_kind,       -- distribution_utility | municipal_multi_service | district_energy_plant |
  onsite_generation_ppa
  ownership,          -- public_municipal | investor_owned | internal_campus_owned | third_party_
  owned
  billing_model,      -- tariff_metered | bundled_municipal | internal_allocation | ppa_generation
  n
  primary_category,   -- primary commodity, for back-compat rollups
  config_ref          -- data/providers/<provider_id>.yaml
)

provider_service(
  provider_service_id,
  provider_id,
  commodity,          -- electric | natural_gas | water | sewer | stormwater | chilled_water | steam | solar_generation
  role,              -- consumption | generation | production | service_fee
  usage_unit,
  demand_unit,
  has_interval,
  interval_minutes
)

account(
  account_id,
  provider_id,
  site_id,
  building_id,
  synthetic_account_number,
  rate_schedule_id,
  effective_start,
  effective_end
)

```

```

meter(
  meter_id,
  account_id,          -- null for owned meters (submeters, production meters, sensors, logical so
  urces, controllers)
  building_id,
  synthetic_meter_number,
  meter_type,
  lifecycle_status,
  replaces_meter_id,
  effective_start,
  effective_end
)

channel(
  channel_id,
  meter_id,
  commodity,
  role,                -- bill facts, no point: billing_usage | billing_demand | derived_billing_u
  sage | flat_service_fee | monthly_allocation
  -- points: interval_usage | interval_demand | interval_generation | interval
  l_generation_demand | interval_export | interval_production | interval_production_demand | submeter_
  interval_usage | handheld_register_reading | handheld_usage_delta | manual_register_reading | manual
  _usage_delta | sensor_observation | equipment_demand | equipment_status | generated_smoke_test
  unit,
  direction,           -- delivered | generated | exported | produced | returned
  interval_minutes,
  point_type,          -- value written to gateway PointType where applicable
  derived_from_channel_id
)

reference_channel(
  reference_channel_id,
  source_kind,         -- weather_station (sensors, historians, controllers and test signals are o
  wned meters)
  source_id,
  commodity,
  role,
  unit,
  interval_minutes,
  point_type
)

gateway_profile(
  gateway_profile_id,
  component_id,        -- AcquiSuiteGateway | MV90Gateway | BACnetGateway | ... | HmrEventsPublish
  executor_key,        -- gateway.acquisuite | gateway.mv90 | gateway.bacnet | ... | hmr.events.pu
  blish
  gateway_type,        -- ACQUISUITE | MDEF | MV9 | BACNET | ... | HMR_EVENTS
  gateway_id,
  gateway_name,
  gateway_guid,
  source_time_zone,
  gateway_time_zone_id
)

gateway_node(
  gateway_node_id,
  gateway_profile_id,
  gw_node_id,
  name,
  is_enabled,
  timestamp_type,
  vendor_values_json   -- component-specific keys, such as Serial Number or BBMD Address
)

gateway_point(

```

```

gateway_point_id,
gateway_node_id,
source_channel_id,      -- channel_id or reference_channel_id
pt_id,
pt_type,
time_zone_id,
interval_minutes,
description,
vendor_values_json      -- component-specific keys, such as DeviceID, Row Number, RecorderID, Chan
nel, ObjectID, Object Type
)

bill(
  bill_id,
  account_id,
  meter_id,
  billing_start,
  billing_end,
  statement_date,
  usage_quantity,
  usage_unit,
  demand_quantity,
  demand_unit,
  total_cost,
  currency,
  bill_status
)

interval_reading(
  channel_id,
  reading_start,
  reading_end,
  quantity,
  unit,
  quality_code
)

scenario_event(
  scenario_event_id,
  scenario_name,
  event_type,
  effective_start,
  effective_end,
  affected_source_id,
  description
)

expected_assertion(
  assertion_id,
  scenario_name,
  assertion_type,
  target_source_id,
  expected_value,
  tolerance,
  unit
)

eem_company(
  eem_company_id,
  display_name,
  company_type,
  scope_json
)

eem_hierarchy_level(
  eem_hierarchy_level_id,

```

```

    eem_company_id,
    level_number,
    description,
    internal_level_nbr,      -- 0 UserDefined, 1 Site, 2 Location, 4 Meter, 5 Point
    user_defined_field,
    font_awesome            -- DBAdmin Awesomefont value, e.g. fa-building
)

eem_system_context(
    system_company_id,      -- -1
    display_name,
    create_in_dbadmin       -- false
)

eem_permission_profile(
    eem_permission_profile_id,
    display_name,
    capabilities_json,
    restrictions_json
)

eem_security_group(
    eem_security_group_id,
    eem_company_id,
    display_name,
    permission_profile_ids
)

eem_user(
    eem_user_id,
    username,
    display_name,
    email,
    title,
    primary_company_id,
    status
)

eem_group_membership(
    eem_security_group_id,
    eem_user_id
)

eem_utility_provider_target(
    eem_utility_provider_target_id,
    provider_id,
    eem_company_id,
    display_name,
    commodity_setup_json,
    rate_schedule_ids_json,
    stage
)

eem_billing_account_shell(
    eem_billing_account_shell_id,
    account_id,
    eem_company_id,
    provider_id,
    synthetic_account_number,
    account_kind,           -- external_account | internal_cost_center | ppa_agreement
    effective_start,
    effective_end,
    create_before_bills
)

eem_meter_target(

```



```

    eem_meter_target_id,
    meter_id,
    eem_company_id,
    parent_location_id,
    source_account_id,
    provider_id,
    display_name,
    meter_target_kind,      -- physical | logical | weather_station | sensor_group | plant_controller
    measure_type,
    has_bills,
    stage2_gateway_profile_ids_json
)

eem_point_target(
    eem_point_target_id,
    eem_meter_target_id,
    source_channel_id,      -- channel_id, reference_channel_id, aggregate id, or baseline id
    display_name,
    point_target_kind,      -- source_channel | derived | weather_index | aggregate | baseline
    point_type,             -- Analog | Digital | Accumulator | AccumulatorDelta | AccumulatorROC
    measure_type,
    unit,
    interval_minutes,
    role,
    stage2_gateway_profile_ids_json
)

eem_weather_station_assignment(
    eem_weather_station_assignment_id,
    eem_company_id,
    location_source_id,
    weather_meter_target_id,
    temperature_point_target_id,
    hdd_point_target_id,
    cdd_point_target_id
)

eem_aggregate_definition(
    eem_aggregate_definition_id,
    eem_company_id,
    aggregate_point_target_id,
    formula,
    unit,
    interval_minutes,
    member_point_target_ids_json,
    member_multipliers_json,
    required_member_flags_json
)

eem_baseline_definition(
    eem_baseline_definition_id,
    eem_company_id,
    measured_point_target_id,
    baseline_point_target_id,
    related_reason_type,    -- B
    index_point_target_ids_json,
    stage3_data_shape
)

bill_charge_line(
    bill_charge_line_id,
    bill_id,
    line_no,
    commodity,
    charge_type,            -- fixed | customer | energy | demand | tier | tou | seasonal | rider | sur
                             charge | tax | minimum | maximum | credit | adjustment

```

```

determinant_ref,      -- optional link to a rate determinant/component
quantity,
unit,
rate_value,           -- $/unit or % where applicable
amount,
currency,
gl_account_code,      -- optional GL coding placeholder (Phase 3 fills UD segments)
is_overhead
)

digital_reading(
  channel_id,
  reading_start,
  reading_end,
  state_value,         -- enumerated state, e.g. 0/1, on/off, occupied/unoccupied
  state_set,           -- name of the enumerated state set
  quality_code
)

note(
  note_id,
  target_table_code,   -- BL bill | METR meter | BA account | location/building
  target_source_id,
  note_text,
  as_of_date,
  author_user_id
)

image(
  image_id,
  target_table_code,
  target_source_id,
  seq,
  image_kind,          -- bill_image | meter_photo | document
  file_ref,
  as_of_date
)

eem_company_calendar(
  eem_company_calendar_id,
  eem_company_id,
  period_year,
  period_month,
  period_start,
  period_end,
  is_present,          -- false = deliberately omitted (negative-test fixture)
  fixture_role         -- normal | import_rejection_gap
)

eem_permission_grant(
  eem_permission_grant_id,
  eem_company_id,
  eem_security_group_id,
  permission_type,     -- $ Library Rates | A Admin | D Points | EX Bill Export | F Functions | MI
                       -- My Import | MU My Upload | TS Task Manager
  permission_object,   -- SiEAppObjects object name / screen / report this grant covers
  access_level,        -- 1 read | 2 edit | 3 approve
  node_scope_source_id, -- optional hierarchy node (site/building/meter) limiting visibility (type
                       -- D)
  derived_from_profile_id -- eem_permission_profile this grant realizes
)

```

Subsystem Entities (Phases 2–6)

These entities extend the model for the dream-customer roadmap. Field-level structure, EEM table mappings, and assertions live in the per-phase specs under `implementation/dream-customer/`.

Phase	New entities	Spec
2 Rates	rate_determinant, rate_block, rate_effective_date, rate_schedule_calendar, rate_ratchet, rate_class_option, rate_meter_assignment	phase2-rate-engine-spec.pdf
3 AP/GL	gl_account, gl_ud_field, gl_ud_value, ap_export_setup, bill_validation_rule, bill_validation_result, accrual	phase3-ap-gl-validation-spec.pdf
4 Tenant	tenant_space, bill_allocation_config, bill_allocation_detail, bill_allocation_map, bill_aggregation_map, pt_allocation_config, pt_allocation_index, bill_gen_config	phase4-tenant-rebilling-spec.pdf
5 Sustainability	emission_factor_source, emission_factor, emission_meter_map, emission_company_config, es_rating_response, weather_regression_result, benchmark_definition	phase5-sustainability-analytics-spec.pdf
6 Operations	project, project_activity, project_event, alarm_point, alarm_event, audit_event, task_schedule, task_log_entry, workflow, mfr_saved_report, mfr_dashboard, mfr_delivery, mfr_delivery_recipient, energyai_insight, hmr_meter, hmr_read_type, hmr_reading, import_source, import_mapping	phase6-operations-engagement-spec.pdf

Artifact Generation

Generated artifacts should include enough metadata to trace back to:

- Repository commit.
- Scenario manifest version.
- Generator version.
- Source database version.
- Scenario names included in the run.

Generated artifacts should be written under `artifacts/` or `out/`, which are intentionally ignored by git.